

Last updated on **Apr 29, 2026**

Reference solutions for machine learning

Databricks provides a rich collection of tools to help you develop machine learning applications. However, finding these tools and understanding how best to apply them to your machine learning problems can be difficult. The reference solutions provided on this page show detailed examples of how you can use Databricks for some common machine learning applications.

- [Image applications](#)
- [Train recommender models](#)
- [Natural language processing](#)

Was this page helpful?

☐ Yes

☐ No

 Ask Genie

Last updated on **Apr 29, 2026**

Reference solution for image applications

Learn how to do distributed image model inference from reference solution notebooks using pandas UDF, PyTorch, and TensorFlow in a common configuration shared by many real-world image applications. This configuration assumes that you store many images in an object store and optionally have continuously arriving new images.

Workflow for image model inferencing

Suppose you have several trained deep learning (DL) models for image classification and object detection—for example, MobileNetV2 for detecting human objects in user-uploaded photos to help protect privacy—and you want to apply these DL models to the stored images.

You might re-train the models and update previously computed predictions. However, it is both I/O-heavy and compute-heavy to load many images and apply DL models. Fortunately, the inference workload is embarrassingly parallel and in theory can be distributed easily. This guide walks you through a practical solution that contains two major stages:

1. ETL images into a Delta table using Auto Loader
2. Perform distributed inference using pandas UDF

ETL images into a Delta table using Auto Loader

For image applications, including training and inference tasks, Databricks recommends that you ETL images into a Delta table with the [Auto Loader](#). The Auto Loader  Ask Genie

data management and automatically handles continuously arriving new images.

ETL image dataset into a Delta table notebook

[Open notebook in new tab](#)

 Copy link for import

```
dbutils.widgets.text("path", "/tmp/flowers/delta", "Output Delta  
table path")  
dbutils.widgets.text("ckpt_path", "/tmp/flowers/ckpt",  
"checkpointLocation path")
```

1. ETL images into a Delta table

[Expand notebook](#) dataset (https://www.tensorflow.org/datasets/catalog/tf_flowers)
hosted under [/databricks-datasets](#)

Perform distributed inference using pandas UDF

The following notebooks use PyTorch and TensorFlow tf.Keras to demonstrate the reference solution.

Distributed inference via Pytorch and pandas UDF notebook

[Open notebook in new tab](#)

 Copy link for import

2. Do distributed model inference from Delta

- Start from the Delta table `/databricks-datasets/flowers/`, which is a copy of the output table of the ETL image dataset in a Delta table notebook.
- Use scalar iterator Pandas UDF to make batch predictions.

```
import io
```

Expand notebook ▾

```
from tensorflow.keras.applications.imagenet_utils import
```

Distributed inference via Keras and pandas UDF notebook

[Open notebook in new tab](#)

 Copy link for import

Do distributed model inference with TensorFlow tf.keras and Delta

- Start from the Delta table `/databricks-datasets/flowers/`, which is a copy of the output table of the ETL image dataset in a Delta table notebook.
- Use scalar iterator Pandas UDF to make predictions

Expand notebook ▾

```
import io
```

Limitations: Image file sizes

For large image files (average image size greater than 100 MB), Databricks recommends using the Delta table only to manage the metadata (list of file names) and loading the images from the object store using their paths when needed.

Is this page
as this page helpful?

☐ Yes	☐ No
Send feedback	

Last updated on **Apr 29, 2026**

Natural language processing

You can perform natural language processing tasks on Databricks using popular open source libraries such as Spark ML and spark-nlp or proprietary libraries through the Databricks partnership with John Snow Labs.

For examples of NLP with Hugging Face, see [Additional resources](#)

Feature creation from text using Spark ML

Spark ML contains a range of text processing tools to create features from text columns. You can create input features from text for model training algorithms directly in your [Spark ML pipelines](#) using Spark ML. Spark ML supports a range of [text processors](#), including tokenization, stop-word processing, word2vec, and feature hashing.

Training and inference using Spark NLP

You can scale out many deep learning methods for natural language processing on Spark using the open-source Spark NLP library. This library supports standard natural language processing operations such as tokenizing, named entity recognition, and vectorization using the included [annotators](#). You can also summarize, perform named entity recognition, translate, and generate text using many pre-trained deep learning models based on [Spark NLP's transformers](#) such as BERT and T5 Marion.

Perform inference in batch using Spark NLP on CPUs

Spark NLP provides many pre-trained models you can use with minimal code. This section contains an example of using the Marian Transformer for machine translation. For the full set of examples, see the [Spark NLP documentation](#).

Requirements

- Install Spark NLP on the cluster using the latest Maven coordinates for Spark NLP, such as `com.johnsnowlabs.nlp:spark-nlp_2.12:4.1.0`. Your cluster must be started with the appropriate Spark configuration options set in order for this library to work.
- To use Spark NLP, your cluster must have the correct `.jar` file downloaded from John Snow Labs. You can create or use a cluster running [any compatible runtime](#).

Example code for Machine Translation

In a notebook cell, install `sparknlp` python libraries:

```
%pip install sparknlp
```

Construct a pipeline for translation and run it on some sample text:

Python

```
from sparknlp.base import DocumentAssembler
from sparknlp.annotator import SentenceDetectorDLModel, MarianTransformer
from pyspark.ml import Pipeline
```

```
document_assembler =
DocumentAssembler().setInputCol("text").setOutputCol("document")
```

```
sentence_detector =
SentenceDetectorDLModel.pretrained("sentence_detector_dl", "xx") \
    .setInputCols("document").setOutputCol("sentence")
```

```
marian_transformer = MarianTransformer.pretrained() \
    .setInputCols("sentence").setOutputCol("translation")
```

```
pipeline = Pipeline().setStages([document_assembler, sentence_detector,
marian_transformer])
```

```
data = spark.createDataFrame([["You can use Spark NLP to translate text."],
```



Ask Genie

```
" + \
                                "This example pipeline translates English
to French]]).toDF("text")

# Create a pipeline model that can be reused across multiple data frames
model = pipeline.fit(data)

# You can use the model on any data frame that has a "text" column
result = model.transform(data)

display(result.select("text", "translation.result"))
```

Example: Named-entity recognition model using Spark NLP and MLflow

The example notebook illustrates how to train a named entity recognition model using Spark NLP, save the model to MLflow, and use the model for inference on text. Refer to the [John Snow Labs documentation for Spark NLP](#) to learn how to train additional natural language processing models.

Spark NLP model training and inference notebook

[Open notebook in new tab](#)

 Copy link for import

Requirements

To use Spark NLP, create or use a cluster with any compatible runtime version (<https://nlp.johnsnowlabs.com/docs/en/install#databricks-support>). Install Spark NLP on the cluster using Maven coordinates, such as

```
com.johnsnowlabs.nlp:spark-nlp_2.12:4.1.0 .
```

```
%pip install sparknlp mlflow
```

Python interpreter will be restarted.

Expand notebook

Healthcare NLP with John Snow Labs partnership

John Snow Labs Spark NLP for Healthcare is a proprietary library for clinical and biomedical text mining. This library provides pre-trained models for recognizing and working with clinical entities, drugs, risk factors, anatomy, demographics, and sensitive data. You can try Spark NLP for Healthcare using the [Partner Connect integration with John Snow Labs](#). You need a trial or paid account with John Snow Labs to try out the commands demonstrated in this guide.

Read more about the full capabilities of John Snow Labs Spark NLP for Healthcare and documentation for use at their [website](#).

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback